

BaSyx DPP API

TEAM 6 · SWE



Nataliia Chubak
Projektleiterin

inf24271@lehre.dhbw-stuttgart.de



Luca Schmoll
Produktmanager

inf24137@lehre.dhbw-stuttgart.de



Magnus Lörcher
Produktmanager

inf24155@lehre.dhbw-stuttgart.de



Manuel Lutz
Testmanager

inf24224@lehre.dhbw-stuttgart.de



Noah Becker
Systemarchitektur

inf24038@lehre.dhbw-stuttgart.de



Fabian Steiß
Technischer Redakteur

inf24138@lehre.dhbw-stuttgart.de



Felix Schulz
UI-Designer

inf24075@lehre.dhbw-stuttgart.de

AGENDA

01 Projektvorstellung

02 Produktübersicht

03 Architektur & Module

04 Vorgehensweise beim Testen

05 Live Demo

06 Fazit & Ausblick

BaSyx DPP API

[README](#) [Code of conduct](#) [MIT license](#)

TINF24F_Team6_BaSyx_DPP_API

[Swagger](#) • [BaSyx Web UI](#) • [Meeting Minutes](#) • [Presentation](#)



Projektvorstellung

Master Usease · Ziele und Kontext

Projektziele

- `</>` REST API Lösung
- Service-Effizient
- Digitaler Produktkatalog für die Produkte
- Digitale Abbildung des Produktlebenszyklus

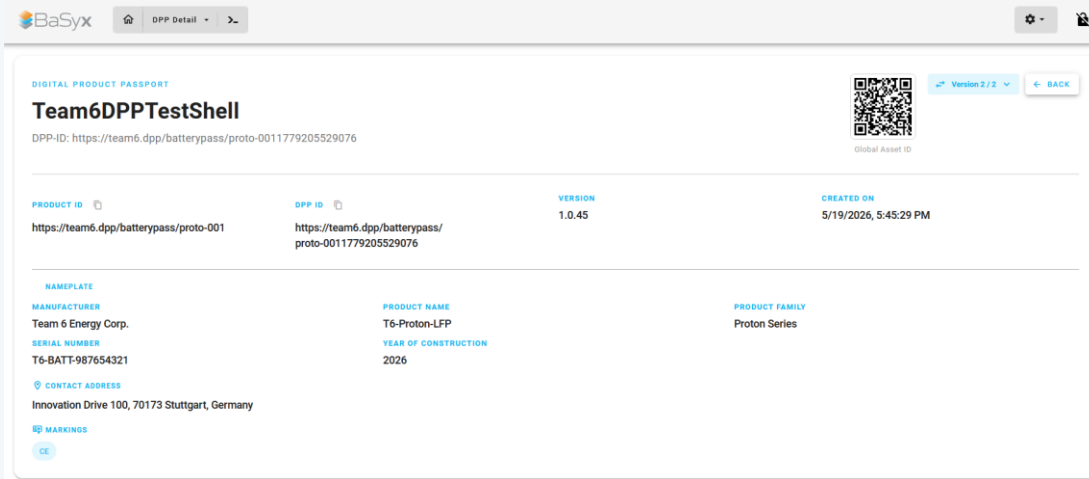


Abb. 1 DPP Viewer (kommt in Live Demo noch mal)

Projektvorstellung

Master Usecase · Ziele und Kontext

FR

Erstellung
eines DPPs

Abrufen
*eines DPPs
per ID*

Updaten
*eines DPPs
per ID*

Löschen
*eines DPPs
per ID*

Abrufen
*eines DPPs
per
productID*

Abrufen
*eines
älteren
DPPs*

NFR

Umsetzung
nach
OpenAPI-
Spezifikation

AAS-
Interaktion
über Standard-
BaSyx-APIs und
-Konnektoren

Automatisierte
DPP-Tests via
Unit- &
Integrations-
tests

DPP API:
Optimiert für
minimale
Lese-Latenz
& Serverlast

Produktübersicht

Frontend-Pages & Backend · Schnittstellen & Funktionen

FRONTEND-PAGES

Nahtlose Integration in BaSyx WebUI

☰ DPP LIST

Auflistung aller AAS

+ Separation in AAS mit DPP und ohne DPP

+ Suchfunktion


AAS Sync
mit AAS
Viewer, etc.

🖨️ DPP VIEWER

Showcase des DPPs einer AAS + Versionierung

Alle 7 relevanten Submodels sichtbar

Visualisierungen

🗄️ DPP REGISTRY

Verwaltung aller DPPs

Nach AAS gegliedert

Löschen & Hinzufügen möglich

BACKEND

Umsetzung als eigenständiger Docker

Microservice in Java

API DATENENDPUNKTE (API)

Implementierung mehrerer Datenendpunkte

- GET
- POST
- DELETE
- PATCH

nach DIN EN 18222 Spezifikation

🗄️ DATENBANK-INTEGRATION

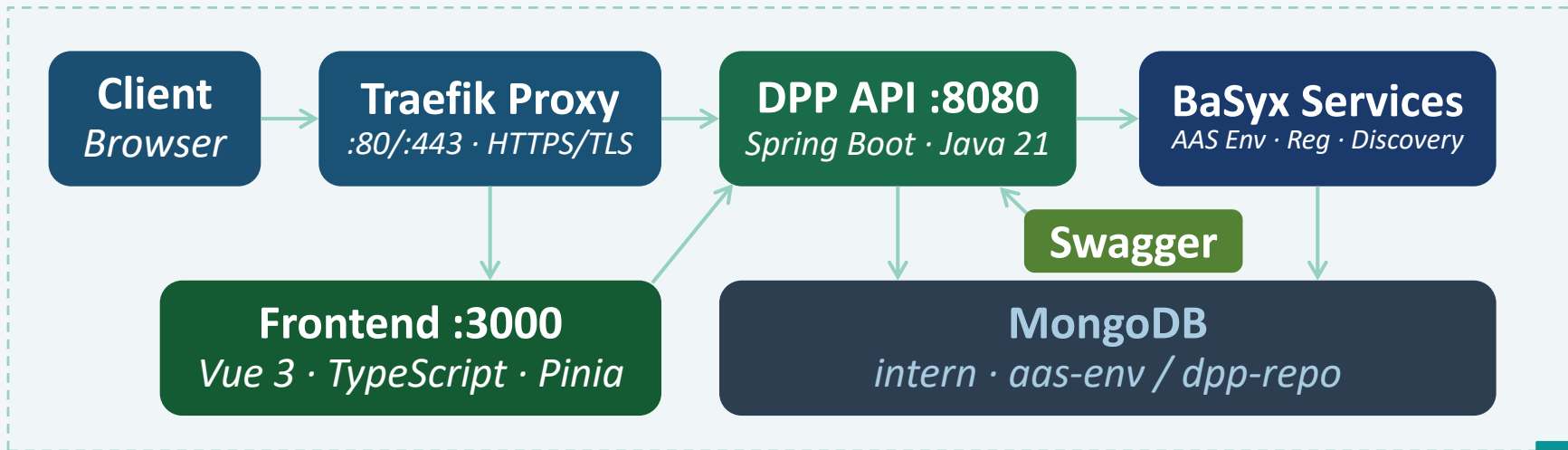
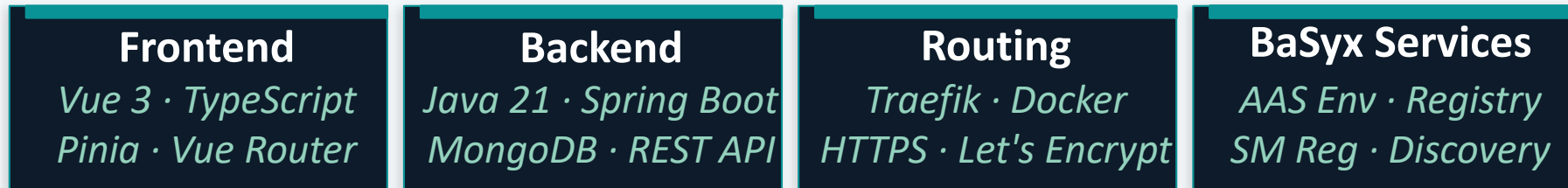
Eigene Verwaltung registrierter DPPs

Speicherung (leicht vereinfacht):

- dppld
- Shell
- Verweise auf Submodels
- Timestamp

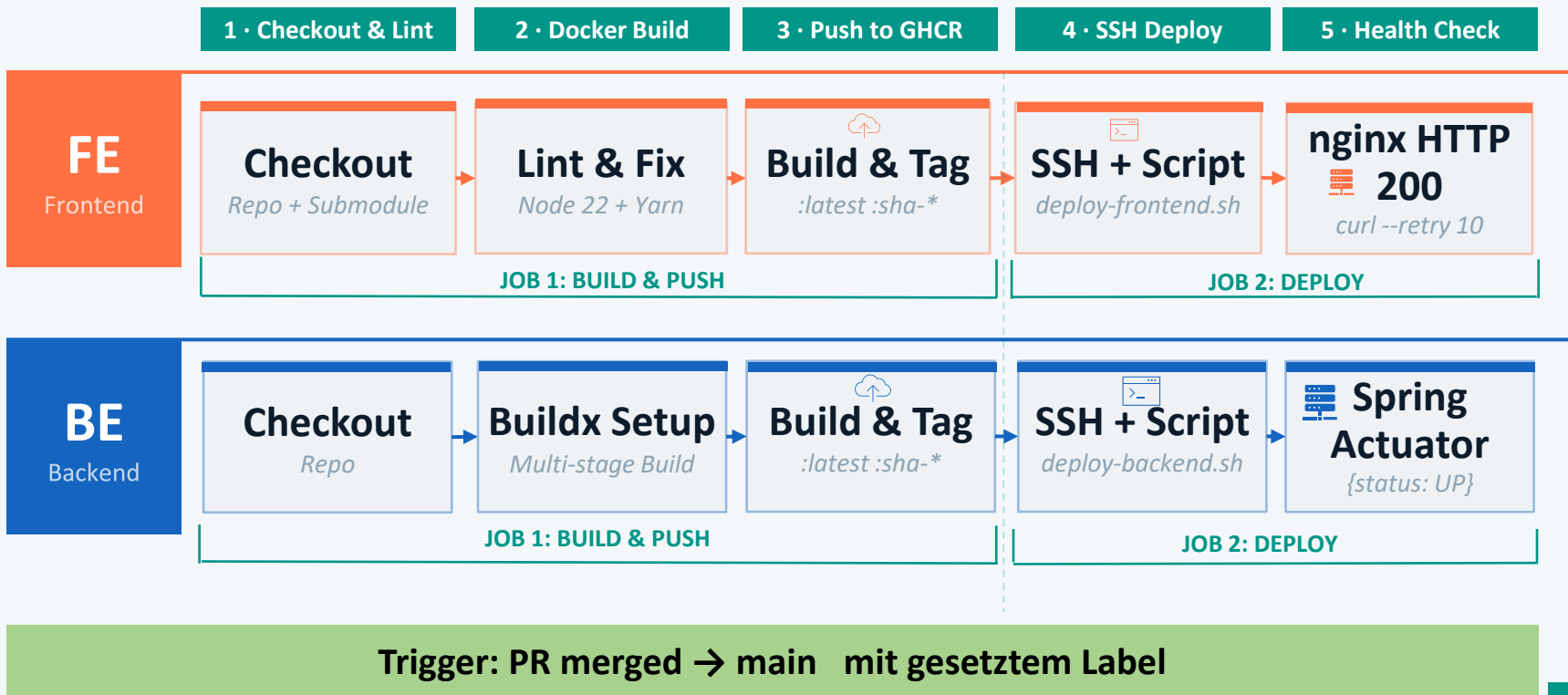
Software-Architektur & -Module

White-Box Sicht · Komponenten & Zusammenspiel



Architektur: Deployment

Server-Deployment-Pipelines via GitHub Actions



DPP Backend - Technologien

DIN EN 18222 konform

Java Spring Boot <i>Java Framework</i>	Modernes Java-Framework für REST APIs, bereits von BaSyx eingesetzt	
Maven <i>Build Tool</i>	Build-Tool & Dependency Management Automatisierte Builds	
OpenAPI 3.0 / Swagger <i>API Spezifikation</i>	API-Dokumentation & Swagger UI	
Eclipse Basyx Java SDK <i>Basyx Integration</i>	Offizielle Integration in die BaSyx-Umgebung Unterstützung von AAS, Submodels & Registry	
MongoDB <i>Datenbank</i>	NoSQL-Datenbank für die Persistenz Verwendet über BaSyx Backend	Architektur Microservices-fähig, integriert in BaSyx-Umgebung Key-Feature Vollständige DIN EN 18222 API-Endpunkte
Docker <i>Containerisierung</i>	Vollständige Containerisierung des Backends Einfaches Deployment mit Docker Compose	

DPP BACKEND

Erstellen

Neue Einträge in MongoDB

Methodik

Mongo connection öffnen
Input dpp in interne Struktur
Mongo Insert call

Methods

Post /dpps
input: dpp json
Patch /dpps/{}/elements{ElementPath}
input: ElementPath

Transformation

Komposition von bestehenden Daten

API Methodik

Eingabe Parsen
Daten von Basyx API / Mongo sammeln
Daten zusammen fassen

Datenbank-Integration

Get /dpps
Submodel-, Administrative Daten
Get /dppsByProductId
DppIds aus der Mongo

Vorgehensweise beim Testen

Testplanung · Testarten · Testergebnisse

Testergebnisse:

- Laufen Größtenteils Automatisch über die GitHub Actions Pipeline
- Bedingung für die Freigabe eines Pull Requests
- Erstellung von GitHub Tasks falls die Tests fehlschlagen

Verwendete Tools & Frameworks:

- GitHub Issues / Test-Tracker
- STP / STR als Dokumentationsgrundlage
- Vitest für automatisierte Tests
- Swagger UI für API-Prüfungen
- Browser DevTools für manuelle Sichtprüfung

Vorgehensweise beim Testen

Testplanung · Testarten · Testergebnisse

Unit/ Integrationstests

- ✓ Mock und Real basiert
- ✓ Umschaltbar über Variable (26 Stück)
- ✓ Aufgeteilt 14 API
- ✓ 7 Front-Backend
- ✓ 5 BaSyx Komponenten

100 %

Abdeckung

Systemtests

- ✓ End-to-End Testfälle
- ✓ Akzeptanztests
- ✓ Manuelle Tests
(25 Stück)

92 %

Abdeckung

Verwendete Tools & Frameworks:

- Vitest · Playwright · Swagger UI · GitHub Issues · Browser DevTools

Fazit & Ausblick

Ergebnisse · Selbstkritik

Fazit & Selbstkritik

Was haben wir erreicht?

- Ein standardisiertes FrontEnd für den digitalen Produktpass
- Ein vollfunktionales Backend für die Verwaltung des DPP

Was hätten wir besser machen können?

- Frühere Kommunikation zum Open-Source Projekt
- Projekt früher starten

Lessons Learned

- KI ist KEIN Wundermittel -> Hilfreich, wenn man es richtig einsetzt
- Kommunikation ist das A & O
- Ein umfängliches Verständnis über die Basisarchitektur ist sehr hilfreich für die Projektumsetzung
- Teambuilding hilft, die Motivation oben zu halten
- Eine gute OpenAPI-Spezifikation erleichtert den Austausch zwischen Frontend und Backend erheblich

Ausblick

Nächster Schritt

1

Unsere Lösung als Branchenstandard etablieren

2

Erweiterung

API Sicherheit implementieren

3

Optimierung

Zugriffszeiten der API optimieren

Vielen Dank!

Fragen & Diskussion